

AI Healthcare Operations & Administration Agent

Project Plan • Technology Stack • Architecture & Flow

Prepared by: Alam Wazir

Role: AI Engineering Mentor / Developer

Purpose: Development plan for review and sign-off

Date: September 2026

This document explains, in plain terms, how the AI Healthcare Operations & Administration Agent will be understood, planned and built — the architecture, the technology stack, the exact flow the agent follows for every request, and the step-by-step build roadmap. It is written so that once this flow is approved, implementation is a matter of execution, not further design.

1. Project Understanding

We are building an **AI-powered healthcare operations and administration agent**. It is **not** a diagnostic or treatment system — its entire scope is administrative work: appointment booking/rescheduling/cancellation, doctor scheduling, billing and insurance status, internal policy lookup, permitted web search, analytics, reporting, staff communication and escalation.

The core idea a user should experience: instead of navigating multiple separate systems, they simply say what they need in plain language. The agent interprets the request, decides what operation is required, calls the right tool(s), verifies the result, remembers context across turns, and replies with a grounded, honest answer.

Core separation of responsibility (the single most important rule in this project):

- The **LLM** reasons about intent and proposes which tool to call.
- **Deterministic application code** (services) actually performs the operation — database reads/writes, calculations.
- **Backend authorization** (outside the LLM) decides whether the requested action is actually allowed

This separation is what turns a chatbot into a safe, production-grade agent: the model is never the database, never the security boundary, and never the transaction engine.

1.1 What the Agent Must Support

- **Patient operations** — view own appointments/doctor schedules, book, reschedule, cancel.
- **Staff operations** — authorized patient lookup, scheduling, billing/insurance, analytics, communication, reports.
- **Knowledge operations** — internal policy RAG and permitted external web search.
- **Operational automation** — multi-tool workflows, reporting, escalation, audit trail.
- **Safety** — clinical/medical requests are explicitly out of scope and must be escalated, never answered as medical advice.

1.2 Two-Stage Development Strategy

Stage A — Build the complete agent first. Synthetic data, no FastAPI, no production auth. Every tool is implemented and unit-tested, then wired into LangChain + LangGraph. The goal is a stable, correctly-reasoning agent before any security layer is added.

Stage B — Secure backend around the proven agent. FastAPI, JWT authentication, RBAC with data scopes, PostgreSQL persistence, human approvals and audit logging are added around the agent without changing its internal reasoning flow.

2. Overall Architecture

```
CLIENT
|
v
FastAPI --JWT--> Authenticated Identity (user_id, role, scope)
|
v
LangGraph Agent (Supervisor)
+-- Planner / Reasoner
+-- Tool Selection
+-- Permission Engine check
+-- Tool Execution
+-- State Management
+-- Confirmation / Approval
|
+--> Patient tools
+--> Doctor / Scheduling tools
+--> Appointment tools
+--> Billing / Insurance
+--> Analytics
+--> Policy RAG
+--> Web Search
+--> Communication
+--> Reports
+--> Escalation
|
v
Services / APIs
+-- PostgreSQL
+-- Vector store (pgvector / Qdrant)
+-- Redis
+-- External providers (mocked in Stage A)
|
v
Audit / Observability
```

During Stage A the FastAPI boundary is intentionally left out — the same agent/tool graph is built and tested with controlled synthetic data first. FastAPI is layered on afterwards as the secure application boundary, not a replacement for the agent's internal workflow.

2.1 Why This Is an Agent, Not a Chatbot

A plain chatbot answers from model knowledge alone. This system is an **agent** because it decides what operation is needed, calls external capabilities, inspects the results, maintains state across turns, performs multiple steps in sequence, recovers from failures, and pauses for confirmation or approval before consequential actions.

```
User request
-> understand intent
-> determine missing information
-> select tool
-> validate arguments
```

```
-> execute tool  
-> observe result  
-> decide next step --> another tool / ask user / confirm+approve / escalate  
-> final answer (only from verified tool results)
```

3. Technology Stack

Technology	Role in the System
Python	Core implementation language
LangChain	Typed tool definitions, prompt/model integration
LangGraph	Stateful agent orchestration, conditional routing, approval pauses
Llama / Qwen via Groq	Fast LLM inference for reasoning and tool selection
FastAPI	Production REST API boundary (Stage B)
PostgreSQL	Transactional persistence — users, patients, appointments, billing, audit
pgvector / Qdrant	Vector store for internal policy RAG retrieval
Redis	Caching, short-lived state, background coordination
JWT	Authentication and identity in every request
Docker / Docker Compose	Reproducible local and deployment environment
pytest	Unit, agent, RBAC and API test suite

External calendar, insurance, communication and web-search providers can initially be mock services with clean interfaces, so real providers can be substituted later without changing the agent or tool logic.

4. Complete Tool Catalog

Every tool has a strict input schema, a predictable output shape, explicit failure states, and a known side-effect level (read-only vs. write vs. consequential-write).

Category	Tool	Purpose
Patient	patient_search	Find a permitted patient record
Patient	patient_profile	Read authorized administrative profile
Patient	patient_history	Read authorized appointment/administrative history
Scheduling	doctor_search	Find a provider by name or specialty
Scheduling	doctor_schedule	Retrieve a provider's working schedule
Scheduling	appointment_search	Find existing authorized appointments
Scheduling	availability_search	Find currently bookable slots
Scheduling	appointment_create	Create an appointment (final availability re-check before write)
Scheduling	appointment_cancel	Cancel an appointment (policy + confirmation + authorization)
Scheduling	appointment_reschedule	Move an appointment (find slot, confirm, transactional update)
Finance	billing_lookup	Retrieve invoice / payment status
Finance	insurance_verify	Verify insurance coverage status
Intelligence	analytics	Deterministic operational metrics (e.g. no-show rate)
Intelligence	policy_rag	Retrieve internal administrative policy evidence
Intelligence	web_search	Retrieve permitted public/external information
Operations	communication	Prepare / send authorized messages
Operations	report_generator	Generate structured operational reports
Operations	escalation	Route a request that needs human intervention
Operations	audit_log	Record an important action or security event

The LLM never receives raw SQL, arbitrary Python execution, or database credentials — the registered tools above are the only capabilities it can request.

5. How a Tool Call Works

- 1. User sends a natural-language request.
- 2. LangGraph stores the conversation and current workflow state.
- 3. The LLM determines intent and what information/action is required.
- 4. The LLM proposes a structured tool call with arguments (JSON).
- 5. The tool schema validates those arguments.
- 6. The permission engine checks identity, role, scope and target record (Stage B).
- 7. If allowed, deterministic service code executes the tool.
- 8. The normalized result is returned into graph state.
- 9. The agent decides: another tool needed? information missing? approval needed? task complete?
- 10. The final answer uses only verified tool results — success is never claimed without a successful result.

5.1 LangGraph State (why it's needed)

Appointment workflows are stateful — a user may name the doctor in one message, pick a time in the next, and confirm in a third. LangGraph persists this across the whole conversation:

```
{
  "conversation_id": "...",
  "user_id": "P1004",
  "role": "PATIENT",
  "intent": "BOOK_APPOINTMENT",
  "doctor_id": "D102",
  "date": "2026-09-07",
  "time_range": "morning",
  "available_slots": ["10:30", "11:30", "12:00"],
  "selected_slot": "11:30",
  "confirmation_required": true,
  "confirmed": false,
  "status": "WAITING_FOR_CONFIRMATION"
}
```

6. Appointment Booking Flow (Primary Example)

Request: "Book me with Dr. Ahmed next Monday morning."

- 1. Classify intent as BOOK_APPOINTMENT.
- 2. Identify the patient from the authenticated session (Stage B).
- 3. Call doctor_search to identify Dr. Ahmed.
- 4. Call doctor_schedule for the requested date.
- 5. Call availability_search for the requested time range.
- 6. Present the actual available slots to the user.
- 7. User selects a slot (e.g. 11:30).
- 8. Store the selection in LangGraph state.
- 9. Ask for explicit confirmation.
- 10. After confirmation, authorization verifies the patient may book this appointment.
- 11. Re-check availability immediately before writing (race-condition guard).
- 12. Call appointment_create.
- 13. Record the action with audit_log.
- 14. Use communication to prepare/send a confirmation.
- 15. Return only the verified appointment details to the user.

```
BOOK_APPOINTMENT
+--> doctor_search --> doctor_schedule --> availability_search
|
v
Present slots --> User selects 11:30 --> Confirmation
|
v
Final availability re-check
+-----+-----+
|                                     |
| available                         conflict
|                                     |
|                                     |
v                                     v
appointment_create                 fetch alternatives
|                                     (never claim success)
v
communication --> audit_log --> success response
```

Critical rule: availability can change between the first search and the final write. The system must re-check immediately before creating the appointment. If the slot has been taken, it must never report success — it offers alternatives instead.

7. Rescheduling and Cancellation Flows

7.1 Reschedule

- Find the existing appointment with **appointment_search** (ask which one, if several match).
- Find the doctor's schedule and available slots for the requested new date.
- Present valid choices; user selects a new slot.
- Check policy if it affects the change.
- Ask for confirmation.
- Re-check the new slot immediately before updating.
- Call **appointment_reschedule**.
- Record the event, send authorized communication.
- Return the verified new appointment details.

7.2 Cancellation

- Find the appointment.
- Retrieve the relevant cancellation rule through **policy_rag** when needed.
- Determine whether the request complies with policy; explain consequences.
- Request confirmation if required.
- Call **appointment_cancel** only after authorization and confirmation.
- Write an audit event and send an authorized cancellation notification.
- Return the actual result.

8. Multi-Tool Workflow Example

Request: "Find patients who missed appointments this month, verify their insurance status, and prepare reminder messages for those whose insurance is still active."

```
analytics (no-show list)
  -> patient retrieval (authorized records)
  -> insurance_verify (each patient)
  -> filter: active insurance only
  -> communication (draft reminders)
  -> approval gate (if bulk communication requires it)
  -> send authorized messages
  -> audit_log
  -> final summary to user
```

This demonstrates dynamic tool composition — the agent is not one giant function; it coordinates several smaller, independently-testable capabilities in sequence.

9. Policy RAG vs. Web Search

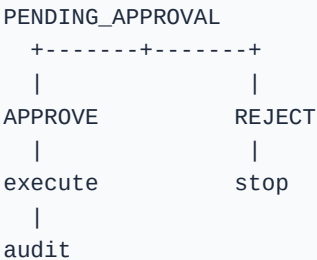
	Policy RAG	Web Search
Scope	Internal organization rules	Public / external information
Examples	Cancellation window, refund rules, working hours, communication rules	Public provider information, general external references
Source	Controlled, versioned internal vector store (pgvector / Qdrant)	Live web search (permitted only)

Policy documents -> Chunk + metadata -> Embeddings -> pgvector / Qdrant
User policy question -> policy_rag(query) -> relevant evidence -> agent workflow

RAG answers internal, organization-specific questions; web search is reserved for permitted public information the organization does not store internally. Neither is ever treated as a medical-advice engine.

10. Human-in-the-Loop

Operation	Typical Control
View own appointments / doctor schedule	Authorization only
Book appointment	Authorization + user confirmation
Cancel / reschedule appointment	Authorization + policy + confirmation
Bulk communication	Authorization + possible staff approval
Sensitive report distribution	Authorization + possible approval
Clinical / sensitive request	Escalation to human review — never answered directly



11. Roles and RBAC

Role	Typical Access	Restriction
PATIENT	Own appointments, own profile, own booking/reschedule/cancel	Cannot access another patient or org-wide data
RECEPTIONIST	Authorized patient lookup and appointment management	Limited to assigned/allowed scope
BILLING / INSURANCE	Billing and insurance operations	Restricted from unrelated protected functions
ADMIN	Organization-wide operational access and reports	Still subject to security and audit controls

Data scopes used throughout: **OWN_DATA**, **ASSIGNED_DATA**, **DEPARTMENT_DATA**, **ALL_DATA**.

11.1 Capability Matrix

Capability	Patient	Receptionist	Billing/Ins.	Admin
Own profile / appointments	Yes	Yes	Yes	Yes
Other patients' data	No	Yes	Limited	Yes
Book appointment	Own only	Yes	No	Yes
Cancel / reschedule	Own only	Yes	No	Yes
Billing	Own / Limited	No	Yes	Yes
Insurance	Own / Limited	Limited	Yes	Yes
Analytics	No	Limited	Limited	Yes
Reports	No	Limited	Yes	Yes
Policy RAG / Web search	Yes	Yes	Yes	Yes
Audit logs	No	No	Limited	Yes

11.2 Why RBAC Must Live Outside the LLM

A prompt instruction is never a security boundary — the model can misunderstand or be manipulated. Authorization is a deterministic function the backend calls, independent of what the LLM "thinks":

```
authorize(user, tool_name, scope, arguments) -> ALLOW / DENY
```

```
P1004 + PATIENT + appointment_search + OWN_DATA + patient_id=P1004 => ALLOW
P1004 + PATIENT + appointment_search + patient_id=P2008             => DENY
PATIENT + ALL_DATA                                                  => DENY
```

The model proposes a tool call; the application alone decides whether it is permitted.

12. FastAPI Production Boundary (Stage B)

```
CLIENT -> POST /api/chat -> JWT validation -> trusted identity (user_id, role, scope)
  -> LangGraph agent -> tool request -> permission engine
    +--> DENY -> safe refusal (no data leak)
    +--> ALLOW -> service/tool -> PostgreSQL / vector store / Redis -> result
  -> LangGraph -> final response
```

12.1 Suggested API Endpoints

Endpoint	Purpose
POST /api/auth/login	Authenticate and issue JWT
POST /api/auth/refresh	Refresh access token
GET /api/me	Current identity and role
POST /api/chat	Natural-language agent endpoint
GET /api/patients/{id}	Authorized patient data
GET /api/appointments	Authorized appointment search
POST /api/appointments	Authorized booking
PATCH /api/appointments/{id}	Authorized rescheduling
DELETE /api/appointments/{id}	Authorized cancellation
GET /api/doctors	Doctor search
GET /api/doctors/{id}/schedule	Doctor schedule
GET /api/reports/{id}	Authorized report retrieval
POST /api/approvals/{id}/approve	Approve a pending action
POST /api/approvals/{id}/reject	Reject a pending action
GET /api/audit-logs	Authorized audit access

12.2 Database Design

Table	Purpose
users	Identity, role and account state
patients	Patient administrative profile
doctors	Provider information
appointments	Appointment records / status
appointment_events	Booking / reschedule / cancellation history
billing_records	Billing information

Table	Purpose
insurance_records	Insurance information
reports	Generated report metadata
approvals	Human approval requests / results
audit_logs	Security / operational event history
policy_documents	Internal policy metadata
conversation_threads	Conversation / LangGraph checkpoint references

13. Project Folder Structure

This is the confirmed structure to build against. Each layer has one job: **agents** = reasoning/state, **tools** = typed capabilities the LLM can call, **services** = real business logic, **api** = HTTP boundary, **auth** = identity + permission engine, **models/database** = persistence, **tests** = correctness at every layer.

```
healthcare-agent/
├── app/
│   ├── main.py                # Application entry point
│   └── api/                   # API routes
│       ├── auth.py
│       ├── chat.py
│       ├── patients.py
│       ├── appointments.py
│       ├── billing.py
│       ├── reports.py
│       └── agents/           # LangGraph agent
│           ├── graph.py
│           ├── state.py
│           ├── planner.py
│           └── nodes/
│               ├── tools/    # Agent tools
│                   ├── patient.py
│                   ├── appointments.py
│                   ├── calendar.py
│                   ├── billing.py
│                   ├── insurance.py
│                   ├── analytics.py
│                   ├── search.py
│                   ├── rag.py
│                   ├── communication.py
│                   ├── reports.py
│                   └── escalation.py
│               ├── services/ # Business logic
│                   ├── patient_service.py
│                   ├── appointment_service.py
│                   ├── billing_service.py
│                   └── insurance_service.py
│               └── auth/     # Authentication & authorization
│                   ├── jwt.py
│                   ├── permissions.py
│                   └── dependencies.py
│               └── models/   # Database models
│                   ├── user.py
│                   ├── patient.py
│                   ├── doctor.py
│                   ├── appointment.py
│                   ├── billing.py
│                   └── audit.py
│               └── database/
│                   ├── connection.py
│                   └── migrations/
│                       └── core/
```

```
■      ■■■ config.py
■      ■■■ logging.py
■■■ tests/
■      ■■■ agent/
■      ■■■ tools/
■      ■■■ services/
■      ■■■ auth/
■      ■■■ api/
■■■ Dockerfile
■■■ docker-compose.yml
■■■ requirements.txt
■■■ .env
```

14. Error Handling and Safety Rules

Situation	Required Behavior
Tool timeout	Retry only when safe; otherwise report unavailable
Slot conflict	Never claim success; fetch and offer alternatives
Invalid identifier	Validate, search, or ask for correction
Unauthorized access	Deny without leaking any protected data
Unknown / unregistered tool	Reject and log — never execute arbitrary code
Web / external API failure	Explain that external information is unavailable
Ambiguous booking request	Ask for the missing information
Database write failure	Never report success
Notification failure after booking	Keep booking result separate from notification status
Clinical request	Escalate / redirect; never diagnose

15. Testing Strategy

Test Layer	What It Covers
Tool unit tests	Valid/invalid arguments and failure cases for every tool
Schema tests	Reject malformed tool calls
Graph tests	Correct state transitions and routing
Booking tests	Successful booking, confirmation, slot conflicts
Reschedule / cancel tests	Policy checks, confirmation, failure paths
Multi-tool tests	Correct sequencing and task completion
RAG tests	Correct policy retrieval and missing-evidence handling
Web search tests	Correct routing to external search
RBAC tests	Every role / scope combination, patient isolation
API tests	JWT validation and status codes
Integration tests	Agent -> permission engine -> service -> PostgreSQL
Regression tests	Every fixed bug becomes a permanent test
Agent evaluation	Tool-selection accuracy, unnecessary calls, task completion, recovery

16. Exact Development Roadmap

Phase 1 — Foundation & Standalone Tools

- Finalize tool contracts and schemas (inputs, outputs, errors, side effects).
- Create synthetic patients, doctors, schedules, appointments, billing and insurance data.
- Implement every standalone tool.
- Unit-test every tool.

Phase 2 — Agent Core

- Connect LangChain tools to the Llama/Qwen model via Groq.
- Create LangGraph state and planner.
- Implement dynamic tool selection and the execution/observation loop.

Phase 3 — Core Workflows

- Implement stateful appointment booking.
- Implement rescheduling and cancellation.
- Implement billing, insurance and analytics workflows.
- Implement policy RAG and web search.
- Implement communication, reports and escalation.

Phase 4 — Hardening & Freeze

- Add retries, validation and safe failure handling.
- Run complete agent tests and evaluation.
- **Freeze the stable agent version.**

Phase 5 — Secure Backend

- Build FastAPI around the stable agent; add PostgreSQL persistence and migrations.
- Add JWT authentication.
- Add RBAC and data scopes.
- Add approvals and audit logging.
- Add Redis / background jobs where needed.

Phase 6 — Deployment

- Dockerize and run integration/security tests.
- Deploy and perform end-to-end verification.

17. Final Project Summary

The final product is an **agentic healthcare administration platform**, not a chatbot. A single natural-language request — "Book me with Dr. Ahmed next Monday morning" — triggers a controlled sequence of real operations: identify intent, find the doctor, inspect the schedule, search availability, remember the selected slot, ask for confirmation, enforce authorization, re-check the slot, create the appointment, communicate the result, and audit the action.

A complex staff request can combine analytics, patient retrieval, insurance verification, communication and reporting in one coordinated workflow. Policy-sensitive questions retrieve internal rules through RAG; external questions use web search. Consequential actions can pause for confirmation or human approval.

17.1 Non-Negotiable Design Rules

- The LLM is not the database.
- The LLM is not the security boundary.
- The LLM is not the transaction engine.
- Never claim an operation succeeded without a successful tool result.
- Re-check time-sensitive resources immediately before committing.
- Separate read operations from write operations.
- Use explicit confirmation for consequential user actions.
- Use human approval where organizational policy requires it.
- Keep internal policy RAG separate from public web search.
- Use synthetic data during development.
- Log important actions and failures.
- Keep workflows deterministic enough to test independently of the LLM.

One-sentence definition: A secure, stateful, tool-using AI healthcare operations agent that understands natural-language administrative requests, dynamically selects and executes the right tools, completes multi-step workflows, respects permissions, handles failures safely, and ultimately operates behind a production FastAPI backend.

Once this flow and structure are approved, the remaining work is implementation against this plan — tool by tool, phase by phase, exactly as outlined above.